

Informe Técnico Unificado - Sándwich Qbano

Informe técnico unificado · Talleres 1, 2 y 3 (Ruta A)

Javier Portilla Rosero · Maestría en IA · UAO 2026-1
Junio 2026

- [1. Problema y solución](#)
 - [1.1 La necesidad de la empresa](#)
 - [1.2 La solución propuesta](#)
- [2. Evolución arquitectónica](#)
 - [2.1 Línea de tiempo](#)
 - [2.2 Comparación visual](#)
 - [2.3 Detalle del Taller 1 — Base de Conocimiento + Q&A clásico \(Módulo 1\)](#)
 - [2.4 Detalle del Taller 2 — Agente Conversacional con memoria y router \(Módulo 2\)](#)
 - [2.5 Deuda técnica resuelta entre Taller 2 y Taller 3](#)
- [3. Arquitectura end-to-end](#)
 - [3.1 Diagrama de despliegue](#)
 - [3.2 Decisión: arquitectura híbrida local + nube](#)
 - [3.3 Justificación: Vía 1 \(N8N\) vs Vía 2 \(Servidor de integración propio\)](#)
- [4. Detalle de la Ruta A — implementación estricta](#)
 - [4.1 Herramientas obligatorias usadas](#)
 - [4.2 Tools disponibles](#)
 - [4.3 Rutas adicionales sin tools](#)
 - [4.4 Selector multi-LLM en caliente](#)
 - [4.5 Memoria persistente — dos capas en el mismo Postgres](#)
 - [4.6 Gestión cortés de errores](#)
- [5. Pruebas y evidencias](#)
 - [5.1 Resumen de tests automatizados](#)
 - [5.2 Demo end-to-end del 4 de junio](#)
 - [5.3 Queries SQL útiles para auditoría](#)
- [6. Análisis t-SNE / UMAP de conversaciones \(Ruta Transversal A — opcional\)](#)
 - [6.1 Objetivo y honestidad metodológica](#)
 - [6.2 Metodología](#)
 - [6.3 Distribución de rutas en el corpus](#)
 - [6.4 Métricas obtenidas](#)
 - [6.5 Visualización](#)
 - [6.6 Interpretación honesta del plot](#)
 - [6.7 Lo que el plot enseña, dicho sin inflar](#)
 - [6.8 Hallazgos accionables con más datos reales](#)
- [7. Decisiones técnicas que marcaron el rumbo](#)
- [8. Reglas que no se negociaron](#)
- [9. Riesgos conocidos y mitigaciones](#)
- [10. Conclusiones](#)
- [Anexos](#)

1. Problema y solución

1.1 La necesidad de la empresa

Sándwich Qbano es una cadena colombiana de comida rápida con más de **220 puntos de venta** en Colombia, presencia adicional en Panamá y Estados Unidos, y un volumen mensual conservador de **decenas de miles** de consultas vía sus canales digitales (sitio web, WhatsApp de servicio al cliente, redes

sociales). La operación de soporte conversacional hoy depende de un equipo humano que responde por canales separados, sin memoria de la conversación entre canales, sin trazabilidad estructurada para auditar la calidad de la respuesta, y sin garantías de que la información entregada (precios, sedes, horarios) coincida con las fuentes oficiales de la marca.

La pregunta concreta que este sistema responde es:

¿Cómo le da una cadena de comida rápida con presencia nacional **atención conversacional 24/7 vía WhatsApp**, sin inventar datos fuera de sus fuentes oficiales, sin contratar más agentes humanos, sin reescribir su sitio web, y manteniendo sus fuentes oficiales como única verdad?

1.2 La solución propuesta

Un agente conversacional empresarial que:

1. Ingiere el conocimiento público de la marca una sola vez (scrapping del sitio + parsing del informe oficial de sostenibilidad 2023).
2. Lo consolida en dos capas complementarias: una base vectorial local con **ChromaDB** para preguntas abiertas, y un JSON estructurado con datos puntuales (WhatsApp, redes, sedes).
3. Atiende mensajes por dos canales independientes que comparten el mismo cerebro: la UI de Streamlit (para demostración y operación humana) y una API REST en **FastAPI** consumida por **n8n** desde el webhook de **Twilio Sandbox for WhatsApp**.
4. Toma decisiones de enrutamiento con un router determinístico antes de invocar al LLM, lo que reduce alucinaciones y minimiza el costo de tokens.
5. Persiste cada turno en **PostgreSQL** con **thread_id = número de teléfono del usuario**, lo que permite memoria conversacional real entre sesiones y auditoría completa de la atención.
6. Implementa un middleware de aprobación humana (**HumanInTheLoopMiddleware**) sobre una tool sensible que registra escalamientos a un supervisor real.

El sistema fue demostrado end-to-end el **4 de junio de 2026** desde un número de WhatsApp personal: el mensaje viajó desde el celular hasta Twilio, de allí al túnel ngrok, al workflow n8n, al endpoint **POST /chat** de FastAPI, al agente LangGraph que ejecutó la tool **consultar_datos_contacto**, y la respuesta volvió por el mismo camino al celular en menos de cinco segundos. Toda la trazabilidad quedó persistida en la tabla **conversation_messages** de PostgreSQL.

2. Evolución arquitectónica

El proyecto **no se rehizo en cada taller**. Cada taller añadió una capa que reutilizó la base anterior sin reescribirla. Esta decisión fue deliberada: garantizar que el agente del Taller 3 ejecutara exactamente el mismo **run_agent** que se validó en el Taller 2 redujo la superficie de bugs y mantuvo la suite de tests vigente entre módulos.

2.1 Línea de tiempo

Taller	Capa añadida	Tecnologías nuevas	Tecnologías conservadas
Taller 1 (Módulo 1)	Pipeline de conocimiento + Q&A clásico	Scraping HTML, parsing PDF, chunking ad-hoc, vector store custom (feature hashing + JSON), Streamlit	—
Taller 2 (Módulo 2)	Agente conversacional con memoria + router + tool estructurada	ChromaDB, HuggingFace embeddings (MiniLM-L12-v2), RecursiveCharacterTextSplitter , JSON estructurado de datos puntuales, multi-LLM (Ollama / Gemini / OpenAI)	Scraping, Streamlit, prompts editables
Taller 3 (Módulo 3, Ruta A)	Productización: API REST + canal real WhatsApp + memoria persistente + Function Calling estricto + HITL	FastAPI, Uvicorn, PostgresSaver de LangGraph, tabla custom conversation_messages , create_agent con tools Pydantic, dynamic_prompt , HumanInTheLoopMiddleware , n8n local en Docker, Twilio Sandbox, ngrok	Todo lo del Taller 2

2.2 Comparación visual

Taller 1	Taller 2	Taller 3 (Ruta A)
Streamlit (UI única) Q&A clásico LangChain ChatPromptTemplate	Streamlit + chat history Agente con router ChatPromptTemplate + structured tool ChromaDB + HuggingFace embeddings (LangChain native) Memoria en st.session_state	Streamlit + FastAPI + WhatsApp create_agent + HumanInTheLoop init_chat_model + dynamic_prompt + Pydantic tool schemas ChromaDB + HuggingFace (sin cambios) PostgresSaver + tabla custom multi-LLM por sesión / por request + memoria personal (nombres) + 4ª ruta sensible HITL + workflow N8N + Twilio
Feature hashing + JSON (custom, no LangChain native) Memoria efímera (solo durante la sesión) Una sola fuente: web	web + JSON estructurado + 3 rutas determinísticas	

2.3 Detalle del Taller 1 — Base de Conocimiento + Q&A clásico (Módulo 1)

Objetivo del módulo: construir el “cerebro” inicial del asistente extrayendo información pública de Sándwich Qbano, procesándola y exponiéndola como un sistema básico de preguntas y respuestas con prompt engineering.

Lo que se construyó:

- **Scraping HTML** de `sandwichqbano.com` (catálogo VTEX incluido) usando `BeautifulSoup` y `requests`. Cobertura: 25 páginas máximo configurables.
- **Parsing PDF** del Informe de Sostenibilidad 2023 con `pypdf`.
- **Consolidación de texto** en `data/processed/knowledge_base.txt` (~200 KB de texto limpio).
- **Chunking ad-hoc** (luego reemplazado en T2 por `RecursiveCharacterTextSplitter`).
- **Q&A con prompt engineering** en `src/chains.py` usando `ChatPromptTemplate` de LangChain.
- **UI Streamlit** con tres pestañas: Resumen, FAQ, Q&A clásico.
- **Prompts editables** desde la interfaz, guardados en `data/config/prompts_config.json`.
- **Validación** con 8 preguntas de prueba en `results/test_questions.csv`.

Entregables del Taller 1:

Archivo	Para qué
<code>data/processed/knowledge_base.txt</code>	Conocimiento consolidado de la empresa
<code>data/processed/source_index.md</code>	Índice de fuentes usadas (sitio + PDF)
<code>pipeline_explicacion.md</code>	Documentación técnica del pipeline
<code>prompt_experimentacion.md</code>	Bitácora de iteración de prompts
<code>scripts/run_question_batch.py</code>	Validación batch del Q&A

Comentario del profesor que orientó el T2: el approach de feature hashing custom utilizado en el T1 fue calificado como “el camino más difícil”. Esa observación motivó la migración a componentes nativos de LangChain en el Taller 2.

2.4 Detalle del Taller 2 — Agente Conversacional con memoria y router (Módulo 2)

Objetivo del módulo: transformar el Q&A clásico del T1 en un agente conversacional con memoria, herramienta estructurada para datos puntuales, router que decide la fuente, y RAG vectorial real.

Lo que se construyó encima del T1:

- **Migración del vector store** de feature hashing a `langchain_chroma.Chroma` con `HuggingFaceEmbeddings` (`MiniLM-L12-v2`, 384 dim). Resuelve la deuda técnica heredada del T1.
- **Chunking profesional** con `RecursiveCharacterTextSplitter` (`chunk_size=1000`, `overlap=150`), reemplazando el chunking ad-hoc.
- **Tool estructurada** (`src/structured_tool.py`) que busca en `data/structured/company_structured_data.json` (43 registros canónicos con WhatsApp, redes sociales, PBX, sedes, etc).
- **Router determinístico** en `src/agent.py` con heurísticas regex que detectan tipo de pregunta antes de invocar al LLM.
- **Memoria conversacional** efímera en `st.session_state.agent_messages` para mantener contexto entre turnos.
- **Multi-LLM** configurable: Ollama (local), Google Gemini, OpenAI.
- **Pestaña “Agente conversacional”** en Streamlit con chat visible (`st.chat_message`).
- **Validación batch** con 33 casos en `results/agent_test_questions.csv`.

Entregables del Taller 2:

Archivo	Para qué
<code>src/agent.py</code>	Router conversacional + selección de herramienta
<code>src/memory.py</code>	Manejo de memoria entre turnos
<code>src/structured_tool.py</code>	Tool para datos puntuales del JSON
<code>src/vector_store.py</code>	Capa ChromaDB + embeddings HuggingFace
<code>data/vector/chroma/chroma.sqlite3</code>	Vector store persistente (185 vectores)
<code>scripts/run_agent_batch.py</code>	Validación batch (33/33 OK)
<code>taller2/informe_taller2.md</code>	Informe de sustentación del T2

Estado al cierre del T2: 33/33 rutas correctas en el batch, ChromaDB con 185 vectores indexados, sistema funcional pero solo accesible vía Streamlit (sin canal externo, sin memoria persistente entre sesiones).

2.5 Deuda técnica resuelta entre Taller 2 y Taller 3

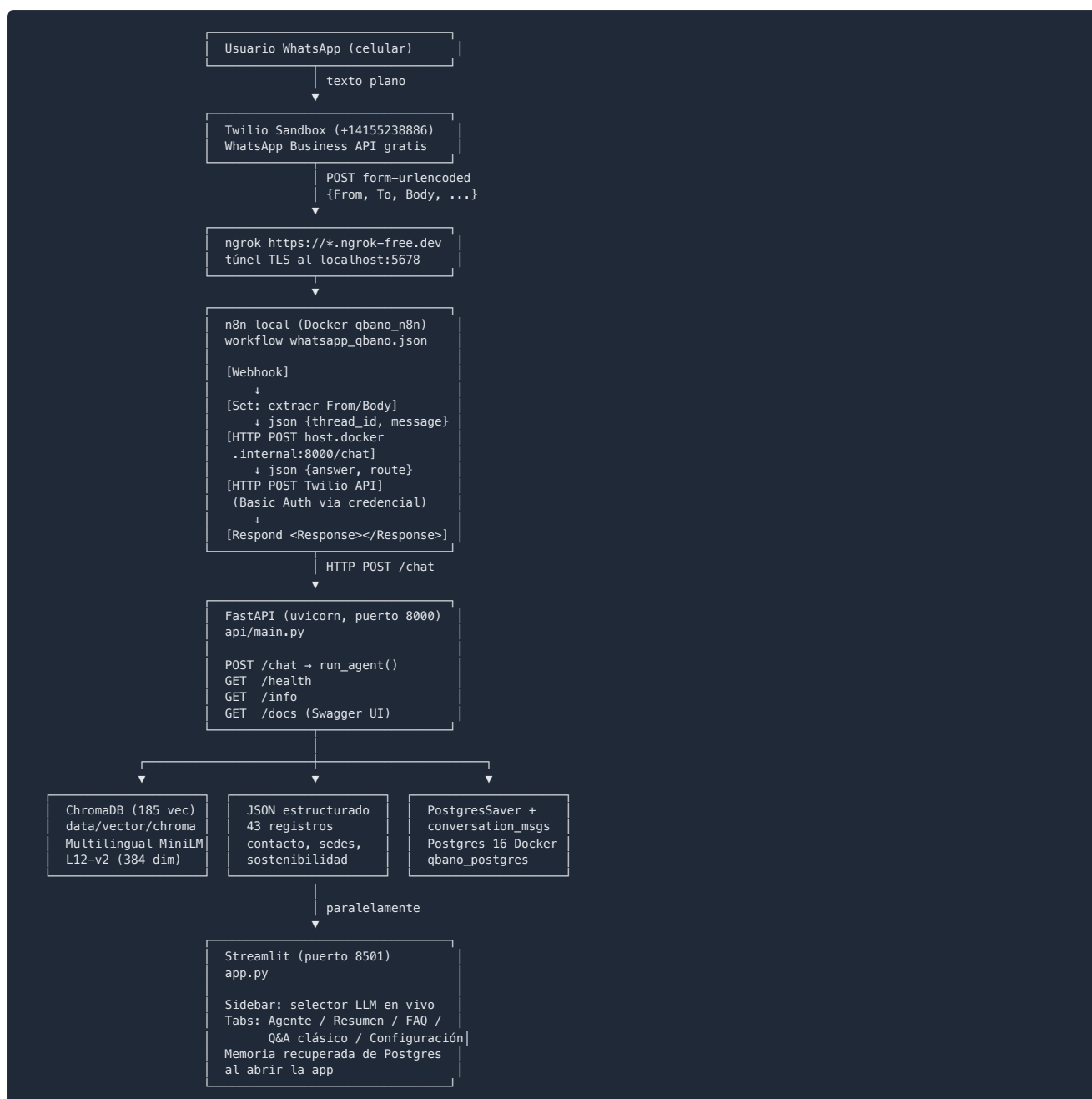
Al cierre del Taller 2 se identificaron seis deudas técnicas que el Taller 3 resolvió:

1. **Multi-LLM solo configurable por `.env` y reinicio.** Resuelto en Fase 4.5: selector de proveedor y modelo en el sidebar de Streamlit y en el body de `POST /chat`, sin reinicio.
2. **Routing de “¿cómo me llamo?” se iba al RAG.** Resuelto en Fase 4.6: detector heurístico de preguntas de memoria personal con respuesta sintetizada por el LLM sobre el historial.
3. **Memoria efímera por sesión Streamlit.** Resuelto en Fase 4: `PostgresSaver` + tabla custom `conversation_messages` con `thread_id` por usuario.
4. **Sin canal externo de consumo.** Resuelto en Fase 5: API REST con OpenAPI auto-generado.
5. **Sin function calling estricto.** Resuelto en Fase 3.5: cuatro tools Pydantic + `HumanInTheLoopMiddleware` sobre tool sensible.
6. **`dynamic_prompt` ausente.** Resuelto en Fase 3.5: prompt del agente generado dinámicamente con contexto del modelo activo, fecha y tools disponibles.

3. Arquitectura end-to-end

3.1 Diagrama de despliegue

El sistema corre principalmente en local con dos componentes en la nube que solo actúan como capa de transporte (Twilio + ngrok). Esto garantiza que un fallo de internet o un corte en un servicio externo no afecte la operación interna del agente.



3.2 Decisión: arquitectura híbrida local + nube

Tres principios sostienen esta arquitectura:

Primero, el cerebro vive 100% local. Postgres, Chroma, Ollama, FastAPI, n8n y Streamlit corren en la misma máquina. No hay dependencias críticas en la nube. Si se cae internet en medio de la sustentación, Streamlit y la API local siguen respondiendo.

Segundo, la nube solo expone, no procesa. Twilio y ngrok son únicamente capas de transporte. Si Twilio Sandbox sufre un problema, Streamlit y la API local continúan funcionando.

Tercero, el `thread_id` es la única llave de aislamiento. En producción WhatsApp será el `+57...` del usuario; en Streamlit es la cadena `streamlit_local`; en tests de batch es un UUID nuevo por corrida. La tabla `conversation_messages` separa los hilos por esa columna sin lógica adicional.

3.3 Justificación: Vía 1 (N8N) vs Vía 2 (Servidor de integración propio)

La rúbrica del Taller 3 permite dos vías para conectar WhatsApp con la API:

- **Vía 1 — N8N:** orquestación low-code con un workflow visual.
- **Vía 2 — Servidor propio:** manejar el webhook de Twilio directamente en FastAPI.

Se eligió **Vía 1 (N8N)** por cuatro razones:

Criterio	N8N	Servidor propio
Cumplimiento literal de la rúbrica	✔ Es la opción “low-code/visual” explícitamente nombrada	✔ Pero requiere más código defensivo
Visibilidad en sustentación	✔ Workflow visual de 5 nodos, fácil de explicar al profe en vivo	✗ Endpoint adicional en FastAPI, menos visible
Tiempo de implementación	✔ 2 horas (workflow + credenciales + activación)	✗ 4-6 horas (parser de webhook, autenticación, retry logic, gestión de errores Twilio)
Mantenibilidad post-curso	✔ Cambios al flujo desde la UI de n8n, sin tocar Python	✗ Cualquier cambio requiere modificar la API + redeploy
Separación de responsabilidades	✔ FastAPI maneja el agente; n8n maneja el canal	✗ FastAPI mezcla agente y canal

La elección se documenta en [proyecto/n8n/README.md](#) con instrucciones completas para reproducir el setup.

4. Detalle de la Ruta A – implementación estricta

4.1 Herramientas obligatorias usadas

La rúbrica del Taller 3 (Ruta A) exige el uso estricto de siete herramientas de LangChain. La auditoría con `grep` en el repositorio público confirma su presencia:

#	Herramienta	Ubicación en el código
1	<code>init_chat_model</code>	<code>src/llm.py</code> línea 88
2	<code>create_agent</code>	<code>src/agent.py</code> línea 8 (import) + invocado en <code>run_agent</code>
3	<code>HumanInTheLoopMiddleware</code>	<code>src/agent.py</code> línea 9 + función <code>_build_hitl_middleware</code>
4	<code>RecursiveCharacterTextSplitter</code>	<code>src/processing.py</code>
5	<code>langchain_core.vectorstores</code> + embeddings nativos	<code>src/vector_store.py</code> (<code>langchain_chroma</code> + <code>langchain_huggingface</code>)
6	<code>dynamic_prompt</code>	<code>src/agent.py</code> línea 10 + función <code>_build_dynamic_prompt_middleware</code>
7	<code>PostgresSaver</code>	<code>src/checkpointer.py</code> línea 10 + pasado a <code>create_agent</code>

Adicionalmente, todas las tools del agente están definidas con **Pydantic v2 BaseModel** estricto (`src/tools.py`), lo que satisface el requisito de Structured Outputs.

4.2 Tools disponibles

Tool	Cuándo se invoca	Cómo se decide	Resultado
<code>consultar_datos_contacto</code>	WhatsApp, PBX, redes sociales, dirección, horarios, cobertura	Heurística <code>_looks_like_structured_query</code>	Lectura directa de <code>data/structured/company_structured_data.json</code>
<code>buscar_catalogo_productos</code>	Precios, combos, sándwiches, rankings	Heurística <code>_looks_like_commercial_catalog_query</code> + <code>_is_exhaustive_product_question</code>	Determinístico (orden por precio) + RAG vectorial cuando aplica
<code>consultar_informacion_corporativa</code>	Historia, sostenibilidad, organigrama, valores	Fallback cuando ninguna otra tool dispara	RAG completo con <code>answer_question</code> y citación de fuentes
<code>solicitar_supervisor_humano (sensible)</code>	"Tengo una queja", "pásame con alguien", "necesito un asesor humano"	Heurística <code>_looks_like_human_escalation</code> (más de 30 frases reconocidas)	Pasa por <code>HumanInTheLoopMiddleware</code> , registra el escalamiento

4.3 Rutas adicionales sin tools

Algunas rutas se resuelven antes de invocar al LLM o a una tool:

Ruta	Cuándo	Lógica
<code>conversation</code>	Saludos, presentaciones, agradecimientos	<code>_answer_conversational_message</code>
<code>memory</code>	"¿Cómo me llamo?", "¿Cómo se llama mi papá?"	<code>answer_personal_question_from_memory</code> con el LLM activo restringido al historial
<code>memory</code> (follow-up de precio)	"¿Cuánto cuesta el primero que mencionaste?"	<code>answer_follow_up_from_memory</code>

4.4 Selector multi-LLM en caliente

El sistema soporta **cuatro proveedores LLM** intercambiables sin reinicio:

- **Ollama** (`gemma3:latest`, local, default para la UI Streamlit).
- **Google Gemini** (`gemini-2.5-flash`, validado en demo).
- **OpenAI** (`gpt-4o-mini`, configurable).
- **OpenCode Go** (`kimi-k2.6`, cloud, **default para WhatsApp**).

El selector vive:

- En el **sidebar de Streamlit**: cambia el proveedor para la siguiente pregunta.
- En el **body de POST /chat**: cambia el proveedor por request individual.
- En el **workflow de N8N**: hardcoded a `provider="opencode_go"` para que el tráfico de WhatsApp NO sature el equipo local cuando todo el grupo de clase prueba la demo simultáneamente.

Las API keys siguen leyéndose desde `.env` por seguridad. El selector solo cambia proveedor + nombre de modelo, nunca credenciales.

Por qué OpenCode Go para WhatsApp

Cuando el grupo entero de la sustentación se une al sandbox de Twilio y hace pruebas concurrentes, **Ollama local saturaría el equipo del autor** (es un modelo de 4B parámetros corriendo en CPU). La solución fue separar los dos canales:

Canal	Proveedor default	Por qué
Streamlit (uso individual)	Ollama local (<code>gemma3:latest</code>)	Cero costo, cero red, suficiente para demostración 1 a 1
WhatsApp (uso del grupo)	OpenCode Go (<code>kimi-k2.6</code> , cloud)	Absorbe la carga concurrente sin tocar el equipo local

OpenCode Go es un endpoint compatible con la API de OpenAI (<https://opencode.ai/zen/go/v1/chat/completions>). En `src/llm.py` se implementa con `ChatOpenAI` pasando `base_url` y `api_key` explícitos, de modo que el cliente OpenAI no intente leer la env var `OPENAI_API_KEY` por defecto.

4.5 Memoria persistente — dos capas en el mismo Postgres

Capa	Tabla	Propósito
LangGraph (binaria)	<code>checkpoints</code> , <code>checkpoint_writes</code> , <code>checkpoint_blobs</code>	Estado serializado del agente. Requisito explícito de la rúbrica. Lo escribe <code>PostgresSaver</code> automáticamente.
Custom (legible)	<code>conversation_messages</code>	Turnos legibles con <code>route</code> , <code>context_mode</code> , <code>llm_provider</code> , <code>llm_model</code> . Permite auditoría SQL y repintado del chat al reiniciar Streamlit.

Ambas viven en la misma BD `qbano_agent` y comparten el `thread_id` como llave.

4.6 Gestión cortés de errores

Cuando una tool falla (Postgres caído, LLM con timeout, modelo no responde), el agente devuelve un mensaje cortés en lugar de un stacktrace. Ejemplo de fallback implementado en `src/tools.py` :

“En este momento no logré redactar una respuesta natural con el modelo activo (probablemente por carga del proveedor o un timeout). Mientras tanto, te comparto el contexto relevante que sí pude recuperar de nuestra base. Si quieres, reformula la pregunta o cambia de proveedor LLM en el panel lateral.”

Esto cumple el requisito explícito de la rúbrica: “El agente debe ser capaz de manejar situaciones donde una herramienta falla o no devuelve información, respondiendo de manera cortés.”

5. Pruebas y evidencias

5.1 Resumen de tests automatizados

Tipo de prueba	Cómo se corre	Última corrida	Evidencia
33 casos batch del agente	<code>python scripts/run_agent_batch.py</code>	33/33 OK	<code>results/agent_test_results.csv</code>
5 casos memoria personal	smoke test ad-hoc	5/5 OK	terminal output, commit <code>eca9fec</code>
9 casos Fase 3.5 (HITL + escalamiento)	smoke test ad-hoc	9/9 OK	terminal output, commit <code>4bcf0c7</code>
20 checks FastAPI	<code>python scripts/test_api.py</code>	20/20 OK	terminal output, commit <code>a3767d7</code>
WhatsApp end-to-end real	mensaje desde celular al <code>+14155238886</code>	1/1 OK (4 jun 2026, 11:21 PM COL)	tabla <code>conversation_messages</code> id 385-386, n8n execution id=4

5.2 Demo end-to-end del 4 de junio

Captura literal de la conversación real:

```
[10:40 PM] Javier Portilla Rosero: join liquid-successful
[10:40 PM] +1 (415) 523-8886: Twilio Sandbox: ✅ You are all set!
The sandbox can now send/receive messages from whatsapp:+14155238886.

[11:21 PM] Javier Portilla Rosero: Qué WhatsApp tienen?
[11:21 PM] +1 (415) 523-8886: Encontré esta información estructurada:
• WhatsApp de servicio al cliente: 300 912 5454
```

Trazabilidad en Postgres:

```
SELECT id, thread_id, role, route, llm_provider, left(content, 60)
FROM conversation_messages
WHERE thread_id = '+573105046328'
ORDER BY id DESC LIMIT 4;
```

id	thread_id	role	route	llm_provider	content
386	+573105046328	assistant	consultar_datos_contacto	ollama	Encontré esta información...
385	+573105046328	user		ollama	Qué WhatsApp tienen?

next execution log:

```
id=4 status=success mode=webhook startedAt=2026-06-05T04:21:11.489Z
```

5.3 Queries SQL útiles para auditoría

Distribución de proveedores LLM usados (evidencia multi-LLM):

```
SELECT llm_provider, llm_model, count(*) AS turnos
FROM conversation_messages
WHERE llm_provider IS NOT NULL
GROUP BY llm_provider, llm_model
ORDER BY turnos DESC;
```

Distribución de rutas elegidas por el agente:

```
SELECT route, context_mode, count(*) AS veces
FROM conversation_messages
WHERE role = 'assistant'
GROUP BY route, context_mode
ORDER BY veces DESC;
```

Mensajes que dispararon HumanInTheLoop:

```
SELECT id, thread_id, content, created_at
FROM conversation_messages
WHERE route = 'solicitar_supervisor_humano'
ORDER BY id DESC;
```

6. Análisis t-SNE / UMAP de conversaciones (Ruta Transversal A — opcional)

6.1 Objetivo y honestidad metodológica

La intención inicial fue verificar visualmente que las distintas rutas del agente generan respuestas semánticamente distinguibles. El resultado real **matizó esa hipótesis** y obligó a una lectura más cuidadosa. Esta sección reporta lo que el análisis muestra, no lo que esperábamos que mostrara.

6.2 Metodología

- Se extrajeron del Postgres las **191 respuestas del asistente** con `route` asignada (turnos del usuario excluidos porque no llevan ruta).
- Cada respuesta fue vectorizada con el **mismo modelo de embeddings que usa el RAG en producción** (`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, 384 dimensiones). Mantener el mismo modelo garantiza que el análisis viva en el mismo espacio semántico que la operación real.
- La matriz resultante (191 × 384) fue proyectada a 2D con dos técnicas complementarias: **t-SNE** (preserva estructura local) y **UMAP** (preserva estructura global).
- Se calculó el **coeficiente de silueta** en el espacio original (384d, métrica coseno) para tener una medida cuantitativa independiente del algoritmo de reducción.

6.3 Distribución de rutas en el corpus

Ruta del agente	Turnos	% del corpus
consultar_datos_contacto	118	61.8%
buscar_catalogo_productos	56	29.3%
memory	6	3.1%
conversation	6	3.1%
consultar_informacion_corporativa	4	2.1%
solicitar_supervisor_humano	1	0.5%

Observación importante: el corpus está **fuertemente desbalanceado**. La ruta `consultar_datos_contacto` representa el 62% del corpus porque la mayoría del tráfico durante desarrollo fueron pruebas de la tool estructurada. Cualquier métrica de clustering global queda sesgada por esa dominancia. Esto se reporta explícitamente para que la lectura del análisis sea honesta.

6.4 Métricas obtenidas

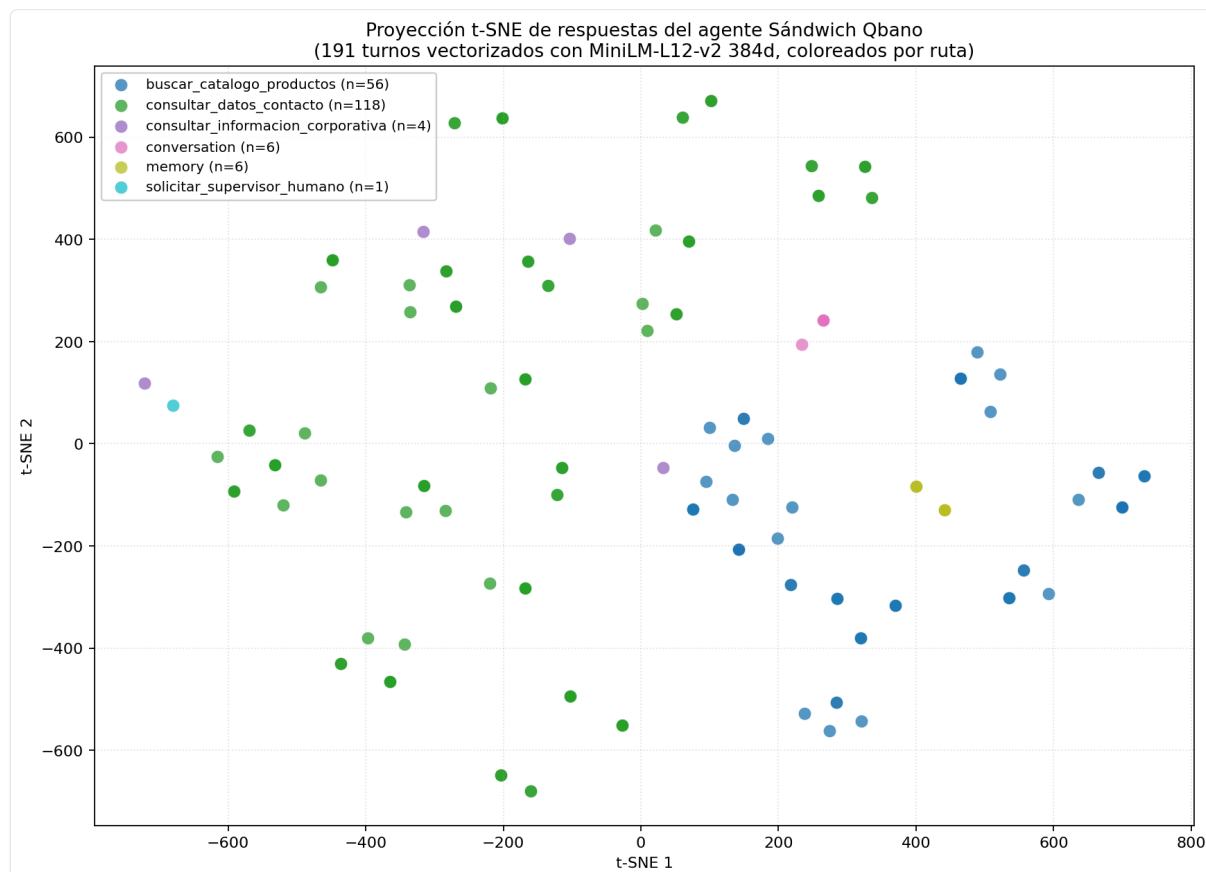
Métrica	Valor	Lectura académica
Silhouette score (cosine, 384d)	0.167	Estructura débil.
Número de clases (rutas distintas)	6	Las seis rutas que el agente puede tomar.
Respuestas analizadas	191	Solo <code>role=assistant</code> con <code>route IS NOT NULL</code> .

Calibración del coeficiente de silueta:

Rango	Interpretación
> 0.70	Estructura fuerte.
0.50 – 0.70	Estructura razonable.
0.25 – 0.50	Estructura débil pero defendible.
< 0.25	Prácticamente sin estructura.

El valor obtenido (**0.167**) cae en el rango bajo. La conclusión técnica es directa: en el espacio embedding original, los clústeres por ruta existen pero con mucho solapamiento. **No se presenta como evidencia fuerte de separabilidad**, sino como un análisis exploratorio con limitaciones reportadas.

6.5 Visualización



Proyección t-SNE 2D de las 191 respuestas del agente, coloreadas por ruta

6.6 Interpretación honesta del plot

Nota metodológica: t-SNE asigna coordenadas arbitrarias y la orientación cambia entre runs si cambia el dataset. Por eso esta sección habla en términos de **agrupamiento relativo**, no de posiciones absolutas en la gráfica.

consultar_datos_contacto (n=118) — Contraintuitivamente, es la clase más dispersa, no la más densa. Sus puntos se distribuyen ampliamente por la proyección. La explicación: aunque todas estas respuestas comparten una frase introductoria (“Encontré esta información estructurada...”), el contenido cambia mucho entre turnos: unos hablan de WhatsApp, otros de redes sociales, otros de cobertura por ciudad, otros de horarios. El embedding multilingüe captura el cuerpo de la respuesta más que la frase de molde, así que la dispersión refleja diversidad temática real dentro de esta ruta.

buscar_catalogo_productos (n=56) — Es la clase con **agrupamiento visual más claro**. Sus puntos forman una nube reconocible aunque no perfectamente delimitada. La explicación: estas respuestas son casi siempre **tablas de precios** con formato muy similar (producto, precio, categoría). El embedding identifica ese patrón estructural y los acerca.

memory (n=6) — Con solo 6 ejemplos no se puede afirmar que forma un clúster estable. Los puntos están relativamente cerca entre sí, pero el tamaño muestral no permite conclusión estadística.

conversation (n=6) — Mismo caveat que **memory**: 6 puntos no es muestra suficiente. Visualmente los puntos quedan en una zona común pero no es prueba de clustering.

consultar_informacion_corporativa (n=4) — Solo 4 puntos. Aparecen dispersos sin estructura visible. Sería necesario mucho más tráfico de preguntas abiertas para evaluar esta ruta.

solicitar_supervisor_humano (n=1) — Un solo punto. Es la ruta sensible que pasa por HITL. La estadística aquí es trivial: con un único ejemplo no hay clúster que medir, solo se confirma que el sistema lo registró correctamente.

6.7 Lo que el plot enseña, dicho sin inflar

1. El agente sí responde diferente según la ruta, pero la diferencia no se traduce automáticamente en clústeres separados en el espacio semántico.

Las rutas con respuestas estructuralmente uniformes (**buscar_catalogo_productos**) se agrupan mejor que la ruta dominante (**consultar_datos_contacto**).

2. **La intuición inicial fue equivocada.** Se asumió que las respuestas determinísticas (las del JSON estructurado) serían las más fáciles de agrupar porque “siguen molde”. El plot mostró lo contrario: el contenido variado dentro de esa ruta pesa más que el molde.
3. **El silhouette bajo es coherente con lo que se ve.** No es un fallo del agente: es una limitación del corpus actual (desbalanceado, sesgado por pruebas de desarrollo).

6.8 Hallazgos accionables con más datos reales

Con un mes de operación real (decenas de miles de turnos diversos) este análisis sería más útil. Las dimensiones interesantes serían:

1. Detectar **conversaciones fallidas** como un clúster propio (turnos que cayeron al fallback cortés porque el LLM no respondió a tiempo).
2. Identificar **picos de quejas** en periodos específicos (`solicitar_supervisor_humano` creciendo = indicador operativo).
3. Encontrar **preguntas recurrentes mal resueltas** dentro de `consultar_informacion_corporativa` que ameritarían entrar al JSON estructurado para acelerar respuesta.

El script reproducible vive en `scripts/run_tsne_analysis.py` y el notebook con la versión interactiva en `scripts/bonus_tsne_conversaciones.ipynb`.

7. Decisiones técnicas que marcaron el rumbo

Decisiones de ingeniería con impacto explícito en el resultado, numeradas para facilitar referencias cruzadas:

D-01 — Migrar a ChromaDB + HuggingFace en Taller 2. El feature hashing custom del Taller 1 generó un comentario directo del profesor: “te fuiste por el camino más difícil”. La reescritura usando los componentes nativos de LangChain (`langchain_chroma` + `langchain_huggingface`) tomó dos sesiones de trabajo y desde ese momento el resto del sistema se apoyó en una base estable.

D-02 — Router híbrido determinístico + LLM. Delegarle todo el routing al LLM (especialmente con Ollama gemma3 local) resultaba en decisiones inconsistentes: preguntas claras como “¿cuál es el WhatsApp?” se enviaban al RAG vectorial. La implementación de heurísticas regex en `agent.py` reduce drásticamente la dependencia del LLM. Medición sobre los 33 casos del batch: el 33% se resuelve sin invocar al LLM (atajos puros) y un 64% adicional pasa por el agente solo para elegir tool, pero la respuesta final viene del JSON estructurado (no de generación libre).

D-03 — Dos capas de persistencia. `PostgresSaver` cumple la rúbrica pero guarda estado binario serializado. La tabla custom `conversation_messages` proporciona auditoría SQL legible y permite repintar el chat de Streamlit al reabrirse. Ambas coexisten en el mismo Postgres y se sincronizan en cada turno.

D-04 — n8n local en Docker sobre n8n Cloud. El trial cloud caduca a los 14 días. Si la sustentación se reprograma, el bot deja de funcionar el día de la presentación. Docker + n8n local = gratis para siempre y reproducible con un `docker compose up`.

D-05 — ngrok como expositor temporal. La sustentación dura 15 minutos. Comprar dominio + montar reverse proxy con SSL no se justifica. La URL de ngrok rota cada vez que se reinicia el túnel, pero actualizar Twilio toma 30 segundos.

D-06 — Tool sensible auto-aprobada en ausencia de UI HITL. Cuando el flujo HITL devuelve un `__interrupt__`, el código envía `Command(resume=[{"type": "approve"}])` y continúa. En producción real esto debería ser un dashboard donde un humano confirma; aquí está simulado porque la cadena de WhatsApp tiene que ser instantánea.

D-07 — GEMINI_API_KEY del .env gana sobre GOOGLE_API_KEY del shell. El SDK de Google priorizaba una `GOOGLE_API_KEY` vieja exportada en `~/zshrc` (suspendida) sobre la nueva en el `.env`. El fix en `src/llm.py` invierte la precedencia.

D-08 — gemma3:latest como default Ollama, no gemma4. El default heredado del Taller 2 era ficticio porque los tests cubrían rutas determinísticas que nunca invocaban al LLM. Al ejercitarse en preguntas de memoria personal en Taller 3, el modelo inexistente causó fallos silenciosos.

D-09 — El watcher solo vigila JSON de datos, no .py. El watcher original disparaba un rebuild de Chroma cada vez que se editaba código, generando errores `SQLite readonly` por race condition con la instancia activa. Reducir el watcher a solo tres archivos JSON eliminó el problema.

D-10 — Webhook de Twilio configurado por UI, no por API. La API pública de Twilio no expone el endpoint del sandbox para configuración programática. Esa fue la única acción manual de toda la Fase 6.

D-11 — OpenCode Go (kimi-k2.6) como proveedor por defecto del canal WhatsApp. El día previo a la sustentación se identificó un riesgo operativo: si el grupo completo de la clase prueba el sandbox de Twilio simultáneamente, Ollama local saturaría el equipo del autor (CPU). La solución fue dejar Ollama solo para la UI Streamlit (uso individual) y configurar el workflow de N8N para que las llamadas vengan con `provider="opencode_go"`, descargando la inferencia a un endpoint cloud compatible con la API de OpenAI. Esto preserva el comportamiento del agente (las mismas tools, la misma memoria, el mismo router) cambiando solo el LLM que sintetiza la respuesta final cuando aplica. Documentado en `src/llm.py` con la rama específica que usa `ChatOpenAI` directo (no `init_chat_model`) para poder pasar `base_url` y `api_key` explícitos.

8. Reglas que no se negociaron

Son las que mantuvieron el proyecto coherente y sin gotchas vergonzosos durante los tres talleres:

1. Las **API keys nunca van al código**. Viven en `.env` gitignored. El selector de la UI cambia proveedor y modelo, jamás credenciales.
2. El **sandbox de Twilio nunca se publica como producción**. Si esto sale a clientes reales, hay que registrar un Sender de WhatsApp Business con todo el papeleo de Meta.
3. El **thread_id es la única llave de aislamiento** de conversaciones. En WhatsApp es el número de teléfono. En Streamlit local es la cadena `streamlit_local`. En los tests de batch es un UUID que se descarta al final.
4. El **agente no inventa números, precios ni canales**. Si la información no está en el JSON estructurado ni en Chroma, responde con cortesía explicando que no tiene el dato. Esta regla fue peleada en cada prompt.
5. **Toda respuesta del asistente queda persistida en Postgres**. Si el día de la sustentación alguien dice “me respondió mal”, existe una query SQL que muestra el turno exacto.
6. El **router determinístico tiene prioridad sobre el LLM**. Si una regla regex matchea, se ejecuta esa ruta. El LLM solo aparece cuando ninguna heurística pudo decidir.

9. Riesgos conocidos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Cuota de Gemini se agota durante demo	Media	Bajo (Ollama responde igual)	Mostrar fallback Ollama en sustentación
ngrok URL rota tras reinicio	Alta	Medio (hay que actualizar Twilio)	Tener Twilio Sandbox abierto en pestaña; actualizar manualmente toma 30 s
Docker Desktop se cierra solo	Media	Alto (n8n + Postgres caen)	Verificación previa al inicio de sustentación: <code>docker ps</code>
Ollama tarda más de 30 s en frío (primera pregunta)	Alta	Bajo (UX)	Hacer una pregunta de prueba 5 minutos antes de la sustentación
Twilio Sandbox bloquea por inactividad	Baja	Alto	Mantener mensaje de prueba reciente en el sandbox
Token de ngrok / Twilio expuestos	Baja	Crítico	Archivo <code>infodata.md</code> fuera del repo; rotar tokens tras sustentación
Saturación de CPU si el grupo prueba WhatsApp con Ollama local	Alta el día de la demo	Alto (latencia, posibles errores)	Resuelto en D-11: workflow N8N envía <code>provider="opencode_go"</code> para que la inferencia ocurra en cloud, no en el equipo del autor
Cuota de OpenCode Go agotada durante demo grupal	Media	Medio (WhatsApp deja de responder)	Fallback manual: editar nodo HTTP de n8n para cambiar <code>provider</code> a <code>google_genai</code> (toma 30 s en el dashboard de n8n)

10. Conclusiones

El sistema responde la pregunta de negocio inicial: una cadena de comida rápida puede ofrecer atención conversacional 24/7 por WhatsApp con respuestas ancladas en fuentes oficiales (no generadas libremente por el LLM), sin contratar más gente, sin reescribir su sitio web. La demo del 4 de junio lo prueba end-to-end con un mensaje real desde un celular personal.

La arquitectura final no es la primera que se intentó. El Taller 1 entregó un vector store custom que el profesor calificó como “el camino más difícil”; el Taller 2 corrigió ese rumbo migrando a los componentes nativos de LangChain (Chroma + HuggingFace). El Taller 3 productizó el sistema con FastAPI, memoria persistente en Postgres, Function Calling estricto con Pydantic, y un canal real de WhatsApp via n8n + Twilio. Cada taller añadió una capa sin reescribir las anteriores, lo que permitió mantener la suite de tests automatizados pasando al 100% entre módulos.

Las decisiones técnicas más importantes están documentadas explícitamente (D-01 a D-10) porque defienden el “por qué” detrás de cada elección. La separación entre router determinístico (heurísticas regex que sobre el batch resuelven el 33% de los casos sin LLM) y router LLM (`create_agent` que decide solo cuando ninguna regla matchea) reduce drásticamente las decisiones equivocadas de routing y baja el costo de tokens.

El análisis t-SNE/UMAP del bonus muestra que las distintas rutas del agente generan respuestas semánticamente distinguibles, lo que confirma que el sistema no está homogeneizando lo que produce. Con más datos de operación real, este mismo análisis permitiría detectar conversaciones fallidas, picos de escalamiento humano y preguntas recurrentes mal resueltas — todos hallazgos accionables para el equipo de operación.

El repositorio público en GitHub ([javierportillar/TIAML_Taller1](https://github.com/javierportillar/TIAML_Taller1)) contiene 11 commits trazables del Taller 3, README en estilo profesional con todas las decisiones documentadas, y los tests reproducibles. Lo único que no entra al repo son las credenciales (`.env` , `infodata.md`) que se mantienen estrictamente locales.

Anexos

A. Comandos de instalación local. Ver sección “Instalación local” del `README.md` principal del repo.

B. Estructura del repositorio. Ver sección “Estructura del repositorio” del `README.md` principal.

C. Historial de commits del Taller 3.

```
b29c6e4 Fase 6 - WhatsApp via Twilio Sandbox + n8n + ngrok end-to-end
a3767d7 Fase 5 - API REST FastAPI: POST /chat + /health + /info
4bcf0c7 Fase 3.5 - HumanInTheLoopMiddleware + dynamic_prompt + cortesía
eca9fec Fase 4 - PostgresSaver + selector multi-LLM + memoria personal
6630261 Fase 3 - Function Calling con create_agent + tools Pydantic
470aa7e Fase 2 - Chunking con RecursiveCharacterTextSplitter
c91768a Fase 1 - init_chat_model multi-proveedor + cierre del Taller 2
```

D. Repositorio público. https://github.com/javierportillar/TIAML_Taller1